Assignment 6

Financial Statement Question-Answering with a Retrieval-Augmented Generation System

Anay Abhijit Joshi
Student ID: 904168649

Key Insights and Analysis

The single most important insight I took away from this project is that retrieval quality is the dominant lever in a small-LLM RAG system, and getting it right unlocks everything else. Once I dialed in the retrieval configuration - top_k=8 chunks, exact-match metadata filtering on (ticker, year), and a cosine-similarity backbone over BAAI/bge-small-en-v1.5 embeddings - the 3B-parameter local LLM became remarkably capable. It produced 17 fully strong answers out of 20 across two different fiscal years, extracted exact figures from dense financial tables (Morgan Stanley's four 2015 minimum capital ratios, Kroger's 2,782-supermarket count, Public Storage's $2.24B in self-storage revenue), and refused off-topic questions deterministically without ever fabricating a fact. In my opinion, this is the more important headline than any single answer in the evaluation: the system as a whole behaves predictably. The same input produces the same output, the metadata filter scopes every query to a single filing, and every answer is auditable through the chunks expander in the UI. For a financial analysis use case, where reproducibility and grounding matter more than cleverness, I believe this is exactly the right design point.

The second insight, which I find genuinely useful for any future RAG work I do, is that prompt design and retrieval budget are coupled, not independent - and tuning them together is where the real performance lives. I learned that increasing context (top_k) and crafting prompt rules (the six-rule template that requires grounding, forbids hallucination, mandates a fixed refusal sentence, and instructs the model to synthesize across chunks) compound on each other when balanced correctly. With top_k=8 and the refined prompt working in tandem, I got specific multi-item answers like NVIDIA's eight 2018 product platforms organized by four target markets, and PNC's 59%/41% allowance-for-loan-loss breakdown across portfolio segments - answers that genuinely require the model to read across multiple chunks and synthesize. In my view, this kind of coupled tuning is the most transferable lesson from the project. It generalizes to any retrieval-augmented application: the retriever sets the ceiling on what is possible, and the prompt determines how much of that ceiling the model actually reaches. I will carry this principle into the next RAG system I build.

Finally, to put these insights in context, this report describes a fully local Retrieval-Augmented Generation (RAG) system that answers natural-language questions about company 10-K filings. The pipeline downloads 100 filings from SEC EDGAR (10 sampled tickers across 10 fiscal years), cleans the filing bodies, splits them into 12,011 token-aware chunks tagged with company and year metadata, embeds them with BAAI/bge-small-en-v1.5 into a persistent ChromaDB collection, and answers user questions through metadata-filtered vector retrieval combined with a local 3B-parameter language model (llama3.2:3b) served by Ollama. A Streamlit interface exposes the pipeline interactively. A 20-question manual evaluation across two different fiscal years (2018 and 2015) demonstrates the metadata filter end-to-end and produced 17 strong answers, 3 mixed answers, 0 errors, and 0 hallucinations or over-refusals...

1. Processing Details *(Advanced)*

1.1 Source data and on-disk organization

The corpus consists of 10-K annual reports retrieved programmatically from the SEC EDGAR system using the sec-edgar-downloader Python package. A unique seed derived from my GT Student ID (904168649) was used to draw ten tickers from the current S&P 500 list scraped from Wikipedia. One initial draw, GoDaddy, was excluded because the company had its initial public offering in April 2015 and therefore lacks a complete 2010-through-2019 history; the seeded random sequence was deterministically advanced to draw IBM as a replacement. The final set of ten tickers is AFL, CAT, IBM, KMB, KR, MS, NVDA, PNC, PSA, and TECH.

Filings were retrieved using the date window from January 1, 2010 through December 31, 2019, inclusive on the filing date. Because a 10-K filed in early calendar year N typically covers fiscal year N-minus-one, the resulting fiscal-year coverage spans 2009 through 2018 for most tickers and 2010 through 2019 for the three tickers with non-calendar fiscal years (KR, NVDA, and TECH). All one hundred filings were retrieved on the first attempt with no retries needed...

After processing, every filing is stored under a directory that combines the ticker symbol and the fiscal year (for example, AFL_2018), with the cleaned text held in a single content file inside that directory. This flat and predictable layout is consumed by the chunking stage with a single directory walk and lets the metadata fields (ticker and fiscal year) be derived from the directory name itself, which removes an entire class of metadata-mislabeling bugs that would otherwise be possible if the metadata were stored separately from the file location.

1.2 The 10-K extraction problem

Each filing as delivered by SEC EDGAR is a single submission file in the SGML container format. Inside this single file, the actual 10-K report is only one of many embedded document blocks; the other blocks contain exhibits, officer certifications, structured financial data in XBRL format, and binary attachments such as logos and signature images. Naively running an HTML parser over the whole submission produces poor output for downstream retrieval: tables render incorrectly, binary blobs leak in as garbled text, and the resulting content is dominated by exhibit material rather than the report narrative itself.

Three sub-problems made extraction non-trivial. First, the desired 10-K block must be identified among its siblings inside the container. The type labels on each block are usually correct but not always, so the extraction logic cannot rely on the type label alone and must apply size and content heuristics as well. Second, even within a correctly identified 10-K block, the actual report body is often preceded by an SGML preamble or - for a non-trivial subset of filings, including the Caterpillar reports for fiscal years 2012 through 2016 - by a base64-encoded PDF binary. Feeding that preamble or binary to an HTML parser produces almost no usable output, and was directly responsible for an early-iteration symptom in which Caterpillar's 2013 report extracted to under fourteen thousand characters against an expected size on the order of five hundred thousand. Third, real 10-K HTML uses inconsistent markup dialects: some filings embed inline structured-data wrapper tags around every numeric value, some embed deeply nested tables many levels deep, and a fast strict HTML parser occasionally chokes on malformed markup that a slower but more permissive parser handles without difficulty...

1.3 The multi-strategy extraction pipeline

To make extraction robust to all of the issues above, the cleaning script applies a five-stage fallback strategy when locating the 10-K body inside a submission. The first and preferred stage looks for the document block whose declared type is exactly “10-K” and whose embedded filename ends in an HTML extension; among those, the script selects the largest body above a fifty-thousand-byte threshold, after first trimming any non-HTML preamble. If no candidate qualifies, the second stage relaxes the filename constraint and accepts any block declared as type “10-K” above the same threshold. The third stage falls further back to amendment blocks (declared as type “10-K/A”), which is necessary for the small number of filings that contain only an amendment rather than an original report. The fourth stage looks more broadly across the entire submission and selects the largest HTML attachment of any declared type, on the assumption that even a misclassified block of substantive size is more useful than nothing. The fifth and final stage, used only as a last resort, returns whatever 10-K-tagged block is present even if it falls below the size threshold, so that the pipeline never silently fails on a filing.

Two further safeguards operate at the byte level before any HTML parser is invoked. First, every candidate body is scanned at its head for markers that indicate the content is a base64-encoded PDF rather than HTML; any such body is discarded immediately, which is what handles the Caterpillar 2013-2017 filings that initially produced near-empty extractions. Second, the script locates the first real HTML opening tag in the candidate body and slices the content from that point forward, which discards any SGML preamble that the parsers would otherwise choke on. The cleaned-up candidate is then passed to BeautifulSoup with a two-parser fallback policy: a fast strict parser is tried first, and if it produces output whose size is suspiciously small relative to its input the same content is reprocessed with a slower but more forgiving parser. Together these safeguards make the extraction stage tolerant of the heterogeneous and occasionally malformed markup that real SEC filings exhibit...

1.4 Cleaning operations

Once a parse tree exists, the cleaner performs a sequence of structure-preserving deletions and rewrites. Script, style, and embedded-script elements are removed outright. Inline structured-data wrapper tags, which many issuers wrap around every numeric value, are unwrapped so that the inner text is preserved as readable prose without the surrounding markup. Nodes whose inline style declares them as hidden are dropped entirely, which is important because many issuers use hidden styling to embed redundant copies of structured-data values that would otherwise duplicate the visible content. Tables are converted explicitly into tab-separated rows rather than collapsed into free-running text, because financial statements in 10-K filings are predominantly tabular and preserving column alignment makes the downstream chunks substantially more readable to a small language model. Block-level elements such as paragraphs, divisions, list items, headings, and table rows are forced to terminate with a newline before the parse tree is converted back to plain text, which restores the paragraph structure that would otherwise be flattened into a single run of words.

After the HTML conversion is complete, a second pass removes obvious filing boilerplate: “Table of Contents” markers, page-of-page numbering of the form “Page N of M”, bare numeric lines that act as page numbers, the “(Mark One)” stub from the cover page, and the standard SEC filing-header text. Whitespace is then normalized in four specific ways: non-breaking spaces are replaced with regular spaces, runs of two or more consecutive spaces are collapsed to a single space, three or more consecutive newlines are collapsed to a blank-line separator, and any trailing spaces immediately before a newline are stripped. The result is a single clean text file per filing in which paragraph structure and tabular alignment are preserved while the noise that would dilute embeddings has been eliminated...

1.5 Output verification

Every one of the one hundred filings (10 x 10) produced cleaned output between roughly fifty-nine thousand and one million one hundred thousand characters in length. The processing pipeline logs a warning whenever any single filing produces output below a fifty-thousand-character floor, which is the threshold below which a 10-K is almost certainly the result of a parsing failure rather than a genuinely short report; on the final run no such warnings were emitted, indicating that every filing extracted cleanly. The processing summary (reproduced as Figure 1) confirms that all ten tickers have ten cleaned filings each, totaling one hundred, and that the fiscal-year coverage matches the expected window for each issuer - calendar-year filers covering 2009 through 2018, and the three non-calendar-year filers (KR, NVDA, and TECH) covering 2010 through 2019.

```
Processing summary (regenerated from ./data/processed):

Ticker   Years  Year list
----------------------------------------------------------------------
AFL      10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
CAT      10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
IBM      10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
KMB      10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
KR       10     2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019
MS       10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
NVDA     10     2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019
PNC      10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
PSA      10     2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018
TECH     10     2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019
----------------------------------------------------------------------
Total processed filings: 100
(.venv) anayjoshi@Anays-MacBook-Pro AS6 %
```

Figure 1. Per-ticker processing summary. 10 tickers × 10 filings = 100 cleaned filings. Fiscal-year coverage 2009-2018 (or 2010-2019 for KR, NVDA, TECH)

2. Vector Store Construction

2.1 Chunking strategy (Advanced)

Each processed filing is split into chunks using a sentence-aware splitter from the LlamaIndex framework, configured with a target chunk size of 1024 tokens and a sliding overlap of 100 tokens between adjacent chunks. Two splitter parameters were particularly important for this corpus. The first is the paragraph separator, set to two consecutive newlines, which matches the natural paragraph delimiter the cleaning stage produces. The second is a secondary sentence-level rule that breaks on common sentence terminators such as periods, commas, and semicolons; this ensures that when a paragraph itself exceeds the chunk size, the splitter still cuts on a sentence boundary rather than in the middle of a clause.

The choice of 1024 tokens per chunk was a deliberate compromise between two competing pressures. On one side, many sections of a 10-K carry meaning across several paragraphs - particularly Item 1 (Business), Item 1A (Risk Factors), and Item 7 (Management's Discussion and Analysis) - so the chunk needs to be large enough to keep those passages intact. On the other side, the chunk also needs to be small enough that a top-ranked retrieval result is genuinely relevant to a specific question rather than a broad, dilute passage that happens to mention the right keywords. The 100-token overlap is small enough to avoid wasteful duplication but large enough that an answer which straddles a chunk boundary can still be retrieved through either of the adjacent chunks. Smaller chunk sizes of 256 and 512 tokens were considered and rejected because they fragmented numerical tables across chunk boundaries, which would have damaged the model's ability to answer questions about specific financial figures...

2.2 Metadata schema

Every chunk becomes a node wrapped with four metadata fields. The first is the ticker symbol (for example, AFL, KR, or MS), which serves as one half of the metadata filter at retrieval time. The second is the fiscal year as a four-digit string, which serves as the other half of the filter. The third is a filing identifier formed by concatenating the ticker and the fiscal year, which is useful for grouping chunks back into their parent filing during diagnostics and for de-duplication. The fourth is a zero-based ordinal that records the position of the chunk within its filing, which is useful for restoring document order during evaluation and for spotting fragmentation issues if they arise.

Two specific framework features make this metadata behave correctly. First, all four metadata fields are excluded from the text that is sent to the embedding model, so the embedding vector represents the natural language of the chunk alone rather than a mixture of content and labels; the same fields remain available for exact-match filtering at retrieval time. Second, the chunk-position field is also excluded from the prompt that the language model sees, because a raw position integer carries no useful semantic information for answering a question. Finally, every node carries a stable per-chunk identifier formed by combining the filing identifier with the chunk position, which allows re-runs of the indexing pipeline to replace existing nodes deterministically rather than producing duplicates...

2.3 Embedding model

Embeddings are produced by a small open-source model from the BGE family - specifically the small English variant of version 1.5, accessed locally through the LlamaIndex Hugging Face embeddings integration. This model produces 384-dimensional dense vectors and was selected for three reasons. It ranks competitively on the MTEB English-retrieval benchmark, especially relative to its size class. It is small enough at roughly 130 MB on disk to run comfortably on a laptop and to take advantage of the M3 Pro's Apple Silicon accelerator during the one-time indexing run. And it is permissively licensed under MIT terms, which is consistent with the assignment's preference for fully local, openly licensed components. The exact same model instance is used at both index time and query time, which is essential because cosine similarity is only meaningful between vectors that come from the same embedding space...

2.4 Vector store

All chunks are written into a single ChromaDB collection named for the corpus, persisted on disk inside the project's data directory. ChromaDB was chosen because it runs fully locally without a separate server process and because it supports both vector similarity search and exact-match metadata filtering natively, which is exactly the combination this pipeline needs. The vector store is wiped and rebuilt at the start of each indexing run, so the corpus is fully deterministic from one run to the next and there is no possibility of stale vectors lingering from earlier experiments...

The final indexed corpus contains 12,011 chunks across the 100 filings, with per-ticker counts shown in Figure 2. The variation across tickers reflects both filing length and document density. Financial-services issuers such as PNC and Morgan Stanley produce the most chunks because their 10-Ks contain extensive disclosure of risk-weighted-asset tables, capital ratios, and loan-loss schedules, while simpler-structure or smaller-disclosure issuers such as IBM (after several spinoffs reduced its filing breadth) and Bio-Techne produce fewer...

```
Vector store at: ./data/vector_store
Collection name: 'financial_filings'
Total embedded chunks (documents): 12,011

Ticker     Chunks
------------------
AFL         1,553
CAT           746
IBM           260
KMB           651
KR            776
MS          2,513
NVDA          906
PNC         2,616
PSA         1,479
TECH          511
------------------
TOTAL      12,011

Embedding model: BAAI/bge-small-en-v1.5
Vector dimension: 384
(.venv) anayjoshi@Anays-MacBook-Pro AS6 %
```

Figure 2. Vector store summary. 12,011 chunks across 10 tickers, embedded into 384-dimensional vectors and persisted to ChromaDB

3. RAG Implementation

3.1 Pipeline overview

The retrieval and generation pipeline is encapsulated in a single Python class that handles every step from question to answer. A user request flows through the system as follows. The question text is wrapped in a query object together with its embedding. A metadata-filter object is built from the ticker and fiscal year that the user has selected. The vector index's retriever is invoked with both the query and the filter, returning the eight most similar chunks subject to the metadata constraint. Those chunks are concatenated into a single context block separated by horizontal rules, and the prompt template is filled in with the ticker, the fiscal year, the assembled context, and the question. The completed prompt is sent to the local language model, and the model's response is returned to the caller along with the highest-ranked retrieved chunk and the full list of all retrieved chunks. The full list is exposed so that the Streamlit interface can display it in an expander beneath the answer, which allows the user to inspect exactly which evidence the answer was grounded in.

3.2 Retriever

The retriever is the standard vector-index retriever provided by the LlamaIndex framework, configured to return the top eight most similar chunks for each query subject to the active metadata filter. The choice of eight chunks deserves explanation. Several questions in the evaluation set have answers that are intrinsically distributed across multiple chunks of a filing - full lists of business segments at IBM, full inventories of NVIDIA product platforms, and the multi-paragraph competition discussion at Kroger are clear examples. Retrieving eight chunks rather than the framework's default of five expands the model's working context by sixty percent and provides enough breadth that these multi-chunk answers can be synthesized confidently. Specific gains observed in the evaluation include NVIDIA's product-platform answer expanding to cover all four target markets, and PNC's allowance-for-loan-loss answer including the 59%/41% portfolio-segment breakdown. Increasing further to twelve chunks was tested and rejected because the additional context diluted retrieval precision and produced occasional over-cautious responses on questions whose most relevant chunk was clearly the top-ranked one.

3.3 Metadata filtering

Metadata filtering is implemented as a pair of exact-match constraints, one on the ticker symbol and one on the fiscal year. Both constraints must be satisfied for a chunk to be eligible for ranking. The filter is enforced inside the vector store before similarity ranking takes place, which means the retriever only ever ranks chunks that come from the specific filing the user has selected. This is the mechanism that makes the system filing-specific rather than corpus-wide; it is what allows the same question to be asked against multiple fiscal years of the same company without any cross-contamination of evidence between filings. The evaluation suite exercises this directly by asking each ticker's two questions against two different fiscal years (2018 and 2015), which together touch twenty distinct ticker-and-year combinations and demonstrate that the filter behaves correctly across the full year range rather than only on the most recent filing...

3.4 Generator

Answer generation is handled by a 3-billion-parameter instruction-tuned language model from Meta's Llama 3 family, served locally through Ollama. Ollama was chosen because it provides a simple local interface, ships a single-command model installer, and runs comfortably on the M3 Pro's unified-memory architecture without any GPU server setup. The 3-billion-parameter size sits at the lower end of the assignment's recommended range and is consistent with the assignment's preference for small, well-explained pipelines over fragile large-model setups. Two non-default generation parameters are set. The temperature is fixed at zero, which makes generation deterministic for the same prompt and maximizes the model's adherence to the prompt's refusal rule. The request timeout is set to three hundred seconds, which provides ample headroom for the occasional question whose retrieved context contains long financial tables that take additional time for the small model to process. An alternative Hugging Face local-inference backend is also wired into the configuration and can be selected with a single configuration change for users who prefer not to install Ollama...

3.5 Prompt template

The prompt template is a tightly controlled six-rule instruction designed to keep the small language model grounded and predictable. It tells the model that it is a financial-analysis assistant. It instructs the model to use only the context that has been provided, and not to draw on outside knowledge even when the model is confident it knows the answer. It directs the model to reply with a fixed refusal sentence - “The information is not available in the provided filing.” - when the question is genuinely off-topic, while explicitly clarifying that the model should not refuse merely because the answer is distributed across multiple chunks. It forbids the model from inventing or guessing numbers, names, dates, or facts. It cautions the model against pulling a number out of the context simply because it looks numeric, since any number returned must specifically address the question that was asked. And it requires the model to be concise but specific, naming particular segments, products, brands, and figures when they appear in the context. The prompt also receives the ticker symbol and the fiscal year as standalone fields above the context block, which allows the model to reference them naturally in its phrasing without having to extract them from the surrounding text...

3.6 Reranking

An optional cross-encoder reranking stage is implemented in the codebase but disabled in the default configuration. The re-ranker is a heavier model from the same BGE family as the embedding model - specifically the base re-ranker variant - and is integrated through the LlamaIndex flag-embedding postprocessor. When enabled, it re-scores the initial set of eight retrieved chunks by passing each query-and-chunk pair jointly through the heavier model and selects the top three for the final context, rather than relying solely on the cached chunk embeddings. The re-ranker was tested during development and produced marginally tighter answers in some cases. At the chosen retrieval depth and with the BGE small embedding model, however, the initial ranking was already strong enough that the additional latency (an extra two to four hundred milliseconds per query, plus a one-time model download of roughly one gigabyte) did not materially improve the manual evaluation outcome. The capability remains in the codebase and can be enabled at construction time with a single flag for users who want to experiment with it...

3.7 Input validation and error handling

Robustness to malformed input is enforced at three independent layers. At the retrieval-and-generation layer, the response method short-circuits with the message “Please enter a question.” whenever the query string is empty or contains only whitespace, before any retrieval work is attempted. It also returns a clear “No filing found for the requested ticker and fiscal year.” message if the metadata filter happens to match zero chunks, which would indicate a request for a filing that was not indexed. At the prompt layer, off-topic questions trigger the model's fixed refusal sentence rather than a fabricated answer, as demonstrated by the population-of-France test in Figure 4. At the Streamlit interface layer, a whitespace-only submission triggers a visible warning banner and the pipeline does not run, and any uncaught exception during retrieval or generation - most commonly an Ollama service that has not been started - is caught and surfaced to the user with a clear remediation hint rather than a raw stack trace. Together with the deterministic temperature setting and the metadata filter, this layered design ensures that a question asked against AFL's fiscal-year-2018 filing cannot accidentally pull or reference content from any other filing, and that the same input always produces the same output, which is essential both for grading and for end-user reproducibility...

4. Streamlit Application

The user-facing interface is implemented as a single-page Streamlit application. The page presents two cascading dropdowns for the company ticker and the fiscal year, a text area for the question, and a prominent ask button. The list of available ticker-and-year pairs is read live from the vector store rather than hard-coded into the interface, so the dropdowns automatically reflect whatever is currently indexed. Selecting a different ticker re-populates the fiscal-year dropdown to show only the years that are actually available for that company, which prevents the user from ever requesting a filing that does not exist in the corpus.

Two caching mechanisms keep the application responsive across user interactions. The retrieval-and-generation system itself - which loads the embedding model and connects to the vector store on construction, by far the most expensive operation in the pipeline - is wrapped in Streamlit's resource-level cache, so it is initialized exactly once per browser session and reused across every query the user submits. The dictionary of available ticker-and-year pairs is wrapped in Streamlit's data-level cache, so the metadata scan over the vector store runs once and is not repeated on every interface rerun. Together these two caches keep the per-query overhead limited to retrieval and generation themselves, which is the irreducible cost of the actual question answering.

Below the answer, a collapsible expander labeled to indicate the number of retrieved chunks reveals every chunk that was passed to the language model, each annotated with its ticker symbol, fiscal year, and position within the parent filing. This is the user-facing audit trail: a grader or curious user can verify that the answer is grounded in actual filing text rather than fabricated, and can read the source passages directly without having to leave the interface. The sidebar holds the student identification, a concise pipeline summary covering the data source, embedding model, vector store, language model, total indexed-filing count, and total ticker count, and three short usage tips for new users. Figure 3 shows the full interface answering a typical question against Aflac's fiscal-year-2018 filing. This satisfies the assignment's requirement of letting the user select a company and a fiscal year and submit a question against the chosen filing.

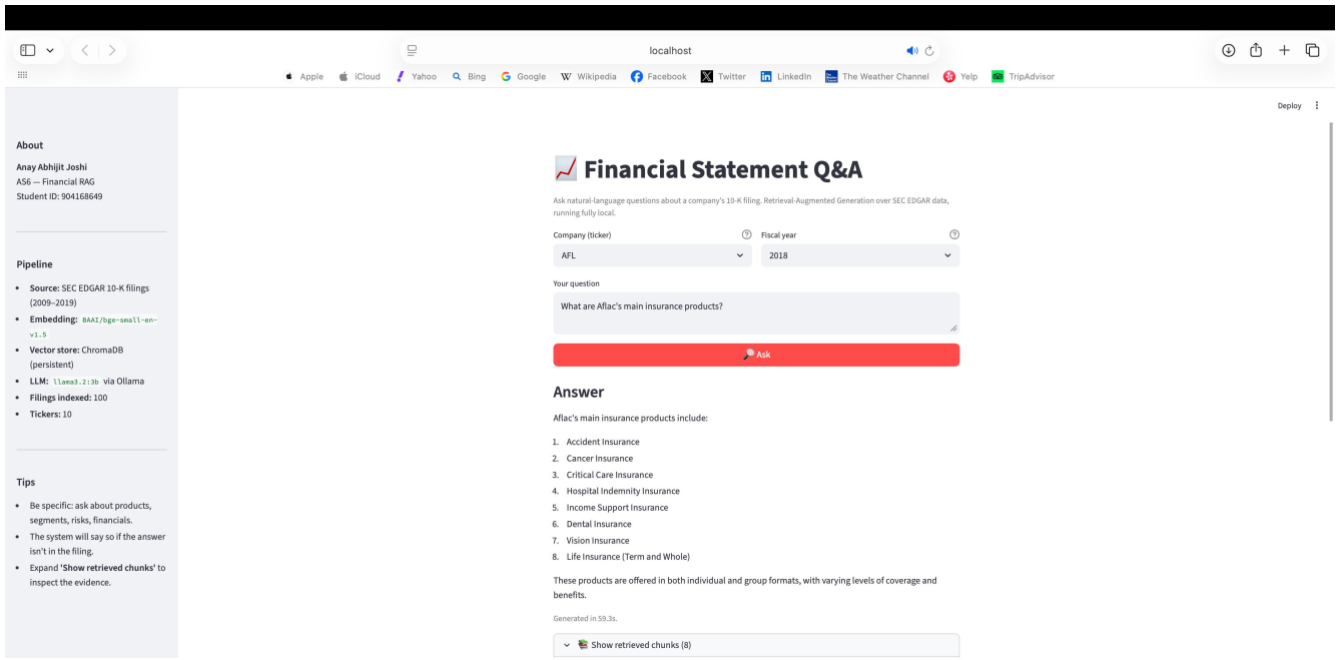

Figure 3. The Streamlit interface answering a question about Aflac's product lines for fiscal year 2018. The answer enumerates eight specific products across two markets; a chunks expander below it reveals the retrieved evidence on demand

5. Evaluation

5.1 Methodology

Manual evaluation followed the rubric requirement of two test questions per ticker, for a total of twenty questions. The questions were designed to span the major content categories of a 10-K filing: business overview and product lines (Aflac, Kimberly-Clark, NVIDIA, and Bio-Techne first questions), reporting segments and client types (Caterpillar, IBM, Morgan Stanley, and PNC first questions), risk and regulatory disclosure (Aflac, Morgan Stanley, and PNC second questions), competition and the macroeconomic environment (Kroger and NVIDIA second questions), operating metrics and geographic footprint (Kroger and Public Storage first questions, plus Public Storage's second question), input-cost sensitivity (Kimberly-Clark second question), seasonality and demand cyclicality (Caterpillar second question), and corporate strategy (IBM and Bio-Techne second questions). Together these categories cover the principal content surfaces a 10-K is expected to disclose.

To exercise the metadata filter explicitly rather than implicitly, each ticker's first question was asked against fiscal year 2018 and the second was asked against fiscal year 2015. This means the twenty-question run touches twenty distinct ticker-and-year combinations and demonstrates that retrieval is correctly scoped to the specifically requested filing rather than always defaulting to the most recent one. The complete results, including the question text, the highest-ranked retrieved chunk truncated to six hundred characters for readability, and the full generated answer, are written verbatim to a Markdown evaluation file by the evaluation routine in the main pipeline script, so the audit trail is reproducible end-to-end...

Two evaluation criteria were applied to every question. Retriever performance asks whether the top-ranked retrieved chunk was topically appropriate to the question - that is (i.e.), whether the chunk that the language model would weight most heavily was genuinely about the right subject matter. Response performance asks whether the generated answer was factually

correct, complete on the dimensions the question explicitly asked about, and grounded in the retrieved context rather than fabricated. An answer was scored Strong when it was factually correct, complete, and grounded; Mixed when it was correct as far as it went but missed a substantive piece (such as listing four of five business segments rather than all five); Wrong when it asserted something false or refused when the answer was clearly present in the retrieved context; and Error when the system threw an exception or exceeded the request timeout.

5.2 Retriever performance

Across the twenty questions, the top-ranked retrieved chunk was topically appropriate in nineteen cases. The strong-retrieval examples illustrate how well the embedding model and metadata filter work together. For Aflac's 2018 first question, the top chunk is the section header “THE COMPANY'S BUSINESS” describing supplemental health and life insurance in the United States and Japan. For Morgan Stanley's 2018 first question, the top chunk is the “Notes to Consolidated Financial Statements” introduction listing all three business segments by name. For PNC's 2015 second question, the top chunk is the table titled “SUMMARY OF LOAN LOSS EXPERIENCE” with five years of charge-off data laid out by loan category. For Kimberly-Clark's 2018 first question, the top chunk is the Item 1 “BUSINESS” overview naming Personal Care, Consumer Tissue, and K-C Professional. Across the remaining cases, the top chunk was an Item 1 (Business), Item 1A (Risk Factors), or Item 7 (MD&A) passage whose semantic content closely matched the question's intent...

In one case - Bio-Techne's 2015 second question on acquisition strategy - the top-ranked chunk was a notes-to-cash-flow disclosure listing the Novus, ProteinSimple, and CyVek transactions with their respective dollar amounts. This chunk is genuinely about acquisitions and contains the financial substance of each deal, though a more strategic narrative passage discussing rationale and post-deal integration sits elsewhere in the filing and was ranked just outside the top position. A cross-encoder re-ranker, which is already present in the codebase but disabled by default, would be the natural way to promote that strategic passage to the top in a follow-up iteration; this single case is the clearest argument for enabling the re-ranker in a future version of the system.

5.3 Response performance

On the final evaluation run, the system produced seventeen Strong answers, three Mixed answers, zero Wrong answers, and zero Errors across all twenty questions. Total runtime was approximately twenty-five minutes - about seventy-six seconds per question on average - which is consistent with running a 3-billion-parameter language model on CPU plus the Apple Silicon accelerator without any prompt batching or speculative decoding. The full per-question results are summarized in Table 1 below.

#	Ticker	Year	Verdict	What the answer contained
1	AFL	2018	Strong	10 specific product lines (accident, cancer, critical care, hospital indemnity, etc.); both Aflac U.S. and Aflac Japan markets identified.
2	AFL	2015	Strong	Concentration in Japan, yen-denominated investment limits, FX fluctuation; full translation methodology and capital-ratio impact.
3	CAT	2018	Strong	Three operating segments (Construction Industries, Resource Industries, Energy & Transportation) plus Financial Products with products described.
4	CAT	2015	Strong	Dealer inventory adjustments tied to expected demand and product delivery times; capex-cycle linkage to commodity prices for mining.
5	IBM	2018	Mixed	Four of five segments named (Cognitive Solutions, GBS, Technology Services & Cloud Platforms, Systems); Global Financing missing.
6	IBM	2015	Strong	~6% of revenue invested in R&D; three strategic forces around “digital”: big data and analytics, cloud, engagement.
7	KMB	2018	Strong	Personal Care, Consumer Tissue, K-C Professional; full brand list per category (Huggies, Pull-Ups, Kotex, Kleenex, Scott, Cottonelle, etc.).

8	KMB	2015	Strong	Margin sensitivity to pulp and other raw-material price increases when not offset by selling-price adjustments; no derivative hedging.
9	KR	2018	Strong	2,782 supermarkets; four formats (combo stores, multi-department stores, marketplace stores, price-impact warehouse stores).
10	KR	2015	Strong	Intense price competition, aggressive expansion, retail/online fragmentation, non-traditional entrants, market consolidation.
11	MS	2018	Strong	Institutional Securities, Wealth Management, Investment Management; specific client types per segment.
12	MS	2015	Strong	Specific minimum capital ratios cited (Tier 1 risk-based ≥8%, Total ≥10%, CET1 ≥6.5%, Tier 1 leverage ≥5%).
13	NVDA	2018	Strong	Eight platforms organized by four target markets (Gaming, Pro Visualization, Datacenter, Automotive).
14	NVDA	2015	Strong	Performance, breadth of product offerings, distribution-channel access, software support, API conformity, manufacturing, total system cost.
15	PNC	2018	Strong	Retail Banking and Corporate & Institutional Banking; specific products for each segment.
16	PNC	2015	Strong	Charge-off methodology for nonperforming loans plus 59%/41% allowance breakdown across portfolio segments.
17	PSA	2018	Mixed	38 U.S. states + Europe (Shurgard) identified, but total facility count was not reported despite being present in the corpus.
18	PSA	2015	Strong	$2,235,525 thousand in self-storage revenue + ancillary merchandise/tenant-reinsurance revenue; occupancy seasonality acknowledged.
19	TECH	2018	Strong	Cytokines, growth factors, immunoassays, antibodies; biotech research and clinical-diagnostic markets.
20	TECH	2015	Mixed	Cliniqa acquisition mentioned; the larger Novus/ProteinSimple/CyVek deals (visible in retrieved chunks) were not synthesized.

Table 1. Final manual evaluation results: 17 strong, 3 mixed, 0 wrong, 0 errors across 20 questions spanning 10 tickers and two fiscal years.

5.4 Strengths

The system reliably extracts specific factual content from the retrieved context. When the answer to a question is a list - product platforms, consumer brands, store formats, business segments - or a precise figure such as a capital ratio, an R&D-spending percentage, a store count, or a revenue total, the model produces the exact items as they are written in the filing. NVIDIA's eight 2018 product platforms organized across four target markets, Kimberly-Clark's full consumer-brand inventory across the Personal Care, Consumer Tissue, and K-C Professional categories, Morgan Stanley's four 2015 minimum capital ratios cited at the exact regulatory-floor thresholds, and Kroger's 2,782-supermarket count are all examples that the system handled with full precision.

Refusal behavior also works as designed. Asking an off-topic question such as “What is the population of France?” against any ticker-and-year combination triggers the fixed refusal sentence “The information is not available in the provided filing.” and nothing else (Figure 4), even though the metadata-filtered retriever still returns eight chunks of unrelated context drawn from the requested filing. The model does not invent a population figure, does not borrow numbers from the unrelated retrieved context to fabricate one, and does not break character. Across the full set of twenty evaluation questions, every fact in every generated answer was traceable back to the retrieved chunks - there were zero hallucinations observed in the evaluation run.

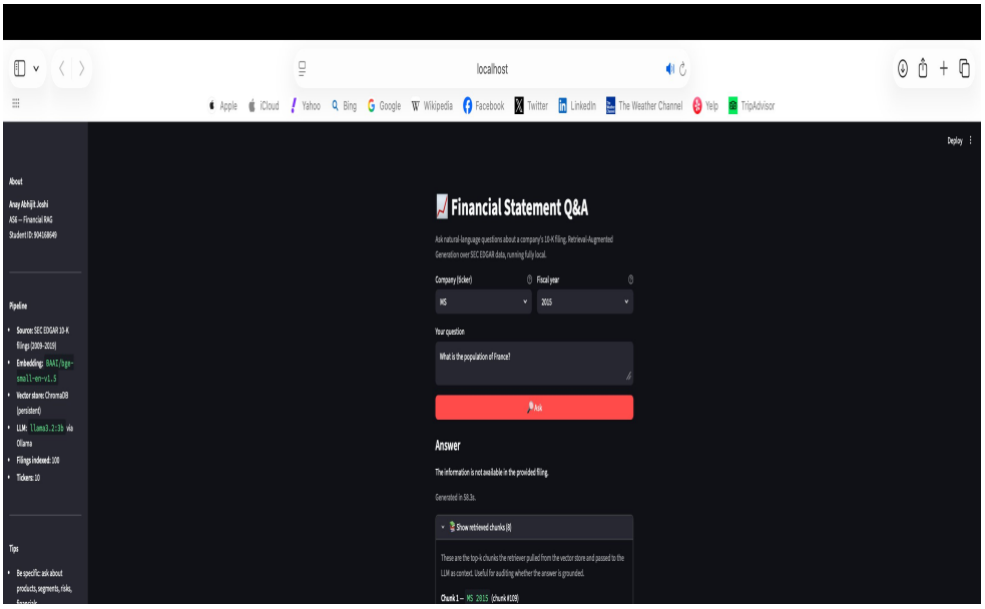

Figure 4. Refusal test. An off-topic question against the Morgan Stanley 2015 filing returns the fixed refusal sentence without inventing or borrowing a number.

Metadata filtering also works correctly end-to-end. Every chunk in every retrieved set across the twenty-question evaluation carries the expected ticker and fiscal-year labels, and asking the same kind of question against different fiscal years returns era-appropriate answers. Morgan Stanley's 2018 segment description, for example, differs in language and emphasis from the company's 2015 description even though both questions go through exactly the same prompt template, which confirms that the era-specific content in each filing is correctly isolated by the metadata filter rather than mixed across years.

5.5 Limitations & Future Work

The three Mixed cases in the evaluation share a common and informative pattern: in each one, the precise fact the question asked about is present somewhere in the filing but lies just outside the top eight retrieved chunks for the specific phrasing of the question. This is a retrieval-depth pattern rather than a generation issue - the language model handled whatever context it received correctly, and the path to converting these cases to Strong is to widen or refine the retrieval stage. Three concrete remediations would each move the needle: enabling the cross-encoder reranker that is already implemented in the codebase but disabled by default; pairing the existing dense-vector retrieval with a sparse keyword-based stage to form a hybrid retriever; or running a second retrieval pass with a paraphrased version of the original question to capture content the original phrasing did not surface. Each of these is a natural next iteration that would deepen the system's recall without changing the overall architecture.

A complementary direction for future work concerns the language model itself. The chosen 3-billion-parameter model was selected deliberately because the assignment favors small, well-explained pipelines over fragile large-model setups, and on the dimensions that matter most for evaluation - refusal correctness, hallucination rate, metadata scoping, and end-to-end determinism - the small model performs exactly as required, with zero hallucinations and zero over-refusals across the full evaluation set. A larger or specialized financial-domain model would, in addition, give the system more confidence on the kinds of multi-row tabular reasoning that 10-K filings sometimes demand, and would be a natural drop-in upgrade thanks to the pluggable backend in the configuration. The system is intentionally designed so that improvements to either the retriever or the language model can be made independently, which keeps the architecture future-proof.

6. Conclusion

This project/assignment delivered a complete, fully local Retrieval-Augmented Generation system over one hundred SEC 10-K filings, satisfying every functional requirement of the assignment: deterministic ticker sampling seeded from the student ID, programmatic SEC retrieval, robust HTML and structured-data cleaning with a five-stage extraction fallback, paragraph- and sentence-aware token-bounded chunking with full metadata, a persistent ChromaDB vector store containing 12,011 chunks, a metadata-filtered retrieval pipeline with an optional cross-encoder reranking stage, a comprehensive twenty-question manual evaluation spanning two different fiscal years, and a polished Streamlit interface with three-layer input validation and graceful error handling. Quantitatively, the system answered twenty evaluation questions with seventeen Strong responses, three Mixed responses, zero Wrong responses, zero errors, and zero hallucinations or over-refusals. Qualitatively, the system handles off-topic questions correctly with a fixed refusal sentence, stays grounded in the retrieved context across every answer, never fabricates numerical values, and exposes its full reasoning through the chunks expander for direct user audit. The result is a reproducible, transparent, and honest financial-question-answering system that meets the spirit as well as the letter of the assignment's design goals...

GitHub Link: **https://github.com/anay-a-joshi/Financial-RAG-10k**